PRACTICAL - 6

Implementation of Matrix Chain Multiplication using Dynamic Programming

Aim of the Practical:

To find the parenthesization with minimum scalar multiplications using dynamic programming and derive its complexity. The program also prints the matrix-cost table, split-position table, every evaluated split, and complete bracket reconstruction.

Recurrence:

$m[i,j] = 0$, if $i = j$

$m[i,j] = \min\{i <= k < j\} [m[i,k] + m[k+1,j] + p[i-1]p[k]p[j]]$

Program:

```
#include <climits>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

string parenthesize(int i, int j, const vector<vector<int>>& split) {
    if (i == j) return "A" + to_string(i);
    int k = split[i][j];
    return "(" + parenthesize(i, k, split) + " x "
               + parenthesize(k + 1, j, split) + ")";
}

void printCostTable(const vector<vector<long long>>& cost, int n) {
    cout << "\nM TABLE (minimum scalar multiplications)\n";
    cout << left << setw(8) << "i\\j";
    for (int j = 1; j <= n; ++j) cout << setw(10) << j;
    cout << '\n';

    for (int i = 1; i <= n; ++i) {
        cout << left << setw(8) << i;
        for (int j = 1; j <= n; ++j) {
            if (j < i) cout << setw(10) << "-";
            else cout << setw(10) << cost[i][j];
        }
        cout << '\n';
    }
}

void printSplitTable(const vector<vector<int>>& split, int n) {
    cout << "\nK TABLE (best split position)\n";
    cout << left << setw(8) << "i\\j";
    for (int j = 1; j <= n; ++j) cout << setw(10) << j;
    cout << '\n';

    for (int i = 1; i <= n; ++i) {
        cout << left << setw(8) << i;
        for (int j = 1; j <= n; ++j) {
            if (j <= i) cout << setw(10) << "-";
            else cout << setw(10) << split[i][j];
        }
        cout << '\n';
    }
}

int main() {
    const vector<int> p = {4, 10, 3, 8};
    const int n = static_cast<int>(p.size()) - 1;

    vector<vector<long long>> cost(n + 1, vector<long long>(n + 1, 0));
    vector<vector<int>> split(n + 1, vector<int>(n + 1, 0));

    cout << "MATRIX CHAIN MULTIPLICATION USING DYNAMIC PROGRAMMING\n";
```

```
    cout << "-------------------------------------------------- \n";
    cout << "Dimension array p = {4, 10, 3, 8}\n\n";
    for (int i = 1; i <= n; ++i) {
        cout << "A" << i << " = " << p[i - 1] << " x " << p[i] << '\n';
    }

    cout << "\nSTEP-BY-STEP DP CALCULATION\n";
    for (int length = 2; length <= n; ++length) {
        cout << "\nChain length = " << length << '\n';
        for (int i = 1; i <= n - length + 1; ++i) {
            int j = i + length - 1;
            cost[i][j] = LLONG_MAX;
            cout << "\nFinding m[" << i << "][" << j << "] for A"
                 << i << "...A" << j << ":\n";

            for (int k = i; k < j; ++k) {
                long long leftCost = cost[i][k];
                long long rightCost = cost[k + 1][j];
                long long multiplyCost = 1LL * p[i - 1] * p[k] * p[j];
                long long total = leftCost + rightCost + multiplyCost;

                cout << "  k = " << k << ": m[" << i << "][" << k << "] + m["
                     << k + 1 << "][" << j << "] + p[" << i - 1 << "]*p["
                     << k << "]*p[" << j << "]\n";
                cout << "         = " << leftCost << " + " << rightCost << " + "
                     << p[i - 1] << "*" << p[k] << "*" << p[j] << '\n';
                cout << "         = " << leftCost << " + " << rightCost << " + "
                     << multiplyCost << " = " << total << '\n';

                if (total < cost[i][j]) {
                    cost[i][j] = total;
                    split[i][j] = k;
                    cout << "         -> New minimum; set m[" << i << "][" << j
                         << "] = " << total << " and k[" << i << "][" << j
                         << "] = " << k << '\n';
                } else {
                    cout << "         -> Not smaller than current minimum "
                         << cost[i][j] << '\n';
                }
            }
        }
    }

    printCostTable(cost, n);
    printSplitTable(split, n);

    string optimal = parenthesize(1, n, split);
    cout << "\nHOW THE BRACKETS ARE FORMED\n";
    cout << "k[1][3] = " << split[1][3]
         << ", so split A1...A3 after A" << split[1][3] << ".\n";
    cout << "Left part A1...A" << split[1][3]
         << " becomes " << parenthesize(1, split[1][3], split) << ".\n";
    cout << "Right part A" << split[1][3] + 1 << "...A3 becomes "
         << parenthesize(split[1][3] + 1, 3, split) << ".\n";
    cout << "Combine both parts: " << optimal << '\n';

    cout << "\nOPTIMAL MULTIPLICATION ORDER\n";
    cout << "Step 1: (A1 x A2) = (4 x 10) x (10 x 3)\n";
    cout << "        Result dimension = 4 x 3\n";
    cout << "        Scalar multiplications = 4*10*3 = 120\n";
    cout << "Step 2: ((A1 x A2) x A3) = (4 x 3) x (3 x 8)\n";
    cout << "        Result dimension = 4 x 8\n";
    cout << "        Scalar multiplications = 4*3*8 = 96\n";
    cout << "Total = 120 + 96 = " << cost[1][n] << '\n';

    cout << "\nFINAL ANSWER\n";
    cout << "Optimal parenthesization: " << optimal << '\n';
    cout << "Minimum scalar multiplications: " << cost[1][n] << '\n';
    cout << "Time complexity: Theta(n^3)\n";
    cout << "Space complexity: Theta(n^2)\n";
    return 0;
}
```

Output:

```
MATRIX CHAIN MULTIPLICATION USING DYNAMIC PROGRAMMING
--------------------------------------------------------------------------------
Dimension array p = {4, 10, 3, 8}

A1 = 4 x 10
A2 = 10 x 3
A3 = 3 x 8

STEP-BY-STEP DP CALCULATION

Chain length = 2

Finding m[1][2] for A1...A2:
  k = 1: m[1][1] + m[2][2] + p[0]*p[1]*p[2]
         = 0 + 0 + 4*10*3
```

```
         = 0 + 0 + 120 = 120
         -> New minimum; set m[1][2] = 120 and k[1][2] = 1

Finding m[2][3] for A2...A3:
  k = 2: m[2][2] + m[3][3] + p[1]*p[2]*p[3]
         = 0 + 0 + 10*3*8
         = 0 + 0 + 240 = 240
         -> New minimum; set m[2][3] = 240 and k[2][3] = 2

Chain length = 3

Finding m[1][3] for A1...A3:
  k = 1: m[1][1] + m[2][3] + p[0]*p[1]*p[3]
         = 0 + 240 + 4*10*8
         = 0 + 240 + 320 = 560
         -> New minimum; set m[1][3] = 560 and k[1][3] = 1
  k = 2: m[1][2] + m[3][3] + p[0]*p[2]*p[3]
         = 120 + 0 + 4*3*8
         = 120 + 0 + 96 = 216
         -> New minimum; set m[1][3] = 216 and k[1][3] = 2

M TABLE (minimum scalar multiplications)
i\j     1         2         3
1       0         120       216
2       -         0         240
3       -         -         0

K TABLE (best split position)
i\j     1         2         3
1       -         1         2
2       -         -         2
3       -         -         -

HOW THE BRACKETS ARE FORMED
k[1][3] = 2, so split A1...A3 after A2.
Left part A1...A2 becomes (A1 x A2).
Right part A3...A3 becomes A3.
Combine both parts: ((A1 x A2) x A3)

OPTIMAL MULTIPLICATION ORDER
Step 1: (A1 x A2) = (4 x 10) x (10 x 3)
        Result dimension = 4 x 3
        Scalar multiplications = 4*10*3 = 120
Step 2: ((A1 x A2) x A3) = (4 x 3) x (3 x 8)
        Result dimension = 4 x 8
        Scalar multiplications = 4*3*8 = 96
Total = 120 + 96 = 216

FINAL ANSWER
Optimal parenthesization: ((A1 x A2) x A3)
Minimum scalar multiplications: 216
Time complexity: Theta(n^3)
Space complexity: Theta(n^2)
```

Matrix Cost Table (m):

m[i,j]	j = 1	j = 2	j = 3
i = 1	0	120	216
i = 2	-	0	240
i = 3	-	-	0

Split Position Table (k):

k[i,j]	j = 1	j = 2	j = 3
i = 1	-	1	2
i = 2	-	-	2
i = 3	-	-	-

Step-by-Step Cost Calculation for the Sample:

- A1 = 4 x 10, A2 = 10 x 3, A3 = 3 x 8.
- m[1,2]: k = 1 gives 0 + 0 + (4 x 10 x 3) = 120.
- m[2,3]: k = 2 gives 0 + 0 + (10 x 3 x 8) = 240.

- m[1,3], k = 1: 0 + 240 + (4 x 10 x 8) = 560.

- m[1,3], k = 2: 120 + 0 + (4 x 3 x 8) = 216 (minimum).

How the Brackets Are Formed:

The value k[1,3] = 2 places the outer split after A2. The left subchain A1...A2 uses k[1,2] = 1, so it becomes (A1 x A2). The right subchain contains only A3. Combining them places the brackets as ((A1 x A2) x A3).

```
A1...A3
  split after A2
  /          \
(A1 x A2)    A3
  \          /
 ((A1 x A2) x A3)
```

Order of Matrix Multiplication:

Step 1: Multiply A1(4 x 10) by A2(10 x 3). The result is 4 x 3 and costs 4 x 10 x 3 = 120 scalar multiplications.

Step 2: Multiply the result (4 x 3) by A3(3 x 8). The final result is 4 x 8 and costs 4 x 3 x 8 = 96 scalar multiplications.

Total cost = 120 + 96 = 216 scalar multiplications.

Time Analysis:

Derivation of Time Complexity:

Let n be the number of matrices. The outer loop chooses chain length, the next loop chooses the starting index, and the inner loop checks every possible split point.

T(n) = Sum(len) Sum(i) Sum(k) O(1)
T(n) = O(n x n x n) = O(n^3)

The cost and split tables each have (n + 1)^2 entries, so S(n) = Theta(n^2).

Final Time Analysis:

- Time Complexity: Theta(n^3).

- Space Complexity: Theta(n^2).

Result:

Matrix Chain Multiplication was implemented successfully. The cost table, split table, evaluated costs, and bracket construction confirm that the optimal parenthesization is ((A1 x A2) x A3), with a minimum cost of 216 scalar multiplications.